

GearGuard: Arsitektur Aplikasi Seluler Agnostik-Detektor untuk Pemantauan Kepatuhan Alat Pelindung Diri (APD)

Nama Penulis

Program Studi, Institusi · Kota, Negara · email@example.com

Abstrak—Memastikan pekerja mengenakan alat pelindung diri (APD) yang diwajibkan sebelum memasuki area kerja adalah tugas keselamatan rutin yang rawan kesalahan. Penelitian yang ada banyak berfokus pada model deteksi, sedangkan alur kerja di sekitarnya—mengidentifikasi pekerja, menerapkan aturan lokasi, mencatat hasil, dan menyusun bukti audit—kurang mendapat perhatian. Makalah ini memaparkan GearGuard, aplikasi seluler lintas-platform yang mencakup alur tersebut secara menyeluruh. Kontribusi kami: (i) definisi formal kepatuhan per-scan dan per-pekerja yang tidak bergantung pada detektor tertentu, (ii) arsitektur berlapis dengan satu antarmuka yang memisahkan model pembelajaran mesin dari antarmuka pengguna, penyimpanan, dan pelaporan, serta (iii) implementasi acuan sumber terbuka berbasis Flutter dengan fitur scan, absensi, peringkat, arsip, dan laporan PDF, yang diverifikasi dengan uji unit dan uji widget menyeluruh. **[BELUM DIISI: satu kalimat hasil utama benchmark detektor dan studi kebergunaan.]**

Kata kunci—alat pelindung diri, keselamatan kerja, aplikasi seluler, deteksi objek, arsitektur perangkat lunak, Flutter

I. PENDAHULUAN

Lokasi konstruksi, manufaktur, dan logistik mewajibkan pekerja memakai helm, rompi reflektif, sarung tangan, dan sepatu keselamatan. Pemeriksaan biasanya dilakukan supervisor di gerbang sehingga lambat, tidak konsisten, dan tanpa catatan terstruktur. Deteksi APD berbasis visi telah banyak diteliti [2] dengan memanfaatkan detektor waktu-nyata seperti YOLO [1]. Namun detektor saja belum menjadi alat keselamatan yang dapat dipakai: supervisor juga perlu tahu *siapa* yang dipindai, aturan *mana* yang berlaku, dan dapat menelusuri riwayat untuk audit.

Kontribusi: (1) formalisasi kepatuhan yang bebas-detektor; (2) arsitektur berlapis yang memperlakukan detektor sebagai komponen yang dapat diganti; (3) implementasi sumber terbuka beserta verifikasinya.

[BELUM DIISI: 3–5 kalimat yang memposisikan penelitian terhadap literatur terkini; verifikasi setiap sitasi.]

II. TINJAUAN PUSTAKA

[BELUM DIISI: survei (a) deteksi APD berbasis pembelajaran mendalam di lokasi konstruksi dan industri, (b) penerapan detektor pada perangkat seluler/edge, (c) sistem manajemen keselamatan dan absensi digital. Gunakan sumber peer-review; nyatakan

celah penelitian: alur kerja menyeluruh dengan detektor yang dapat diganti.]

III. PERUMUSAN MASALAH

Misalkan P himpunan jenis APD yang didukung (helm, rompi, sarung tangan, sepatu, kacamata, masker). Sebuah lokasi menetapkan himpunan wajib $R \subseteq P$, $R \neq \emptyset$, dan ambang keyakinan $\tau \in [0,1]$. Untuk satu frame, detektor mengembalikan keyakinan $c_p \in [0,1]$ bagi setiap $p \in P$. Sebuah item *terdeteksi* bila $c_p \geq \tau$. Himpunan item yang hilang adalah

$$M = \{p \in R \mid c_p < \tau\}. \quad (1)$$

Sebuah scan *patuh* jika dan hanya jika $M = \emptyset$, dengan skor

$$s = (|R| - |M|) / |R|. \quad (2)$$

Untuk pekerja w dengan himpunan scan S_w , tingkat kepatuhan adalah

$$C_w = |\{x \in S_w \mid M_x = \emptyset\}| / |S_w|. \quad (3)$$

dan pekerja diurutkan menurut C_w menurun; bila sama, dipakai rata-rata skor. Tingkat kepatuhan harian lokasi adalah rasio yang sama atas seluruh scan pada hari itu. R dan τ disimpan bersama setiap scan sehingga hasil lama tetap valid ketika aturan lokasi berubah.

IV. DESAIN SISTEM

A. Arsitektur berlapis

GearGuard terdiri atas empat lapisan: *domain* (tipe nilai Dart murni), *data* (detektor, penyimpanan, pembuat PDF), *state* (cubit flutter_bloc dengan state imutabel), dan *presentasi* (modul fitur). Besaran turunan— C_w , peringkat, perlengkapan yang paling sering terlewat—dihitung di lapisan state, bukan di widget, sehingga mudah diuji unit.

presentasi	dashboard scanner attendance team archive settings
state	SafetyCubit SettingsCubit
data	PpeDetector SafetyRepository
report_pdf	
domain	PpeItem Employee ScanRecord

B. Antarmuka detektor

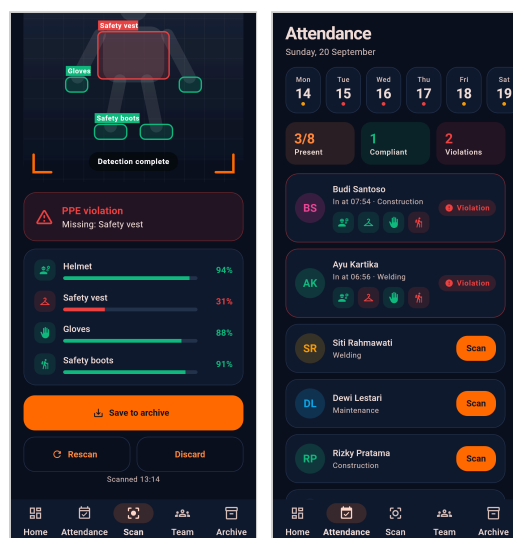
```
abstract interface class PpeDetector {  
    Future<Map<PpeItem, double>> detect();  
}
```

Ini satu-satunya kontrak antara model dan bagian aplikasi lainnya. Antarmuka disuntikkan di akar aplikasi, sehingga mengganti detektor simulasi dengan model pada perangkat

(mis. jaringan YOLO lewat platform channel) tidak memerlukan perubahan pada layar, cubit, ataupun laporan mana pun.

C. Desain interaksi

Navigasi bawah menampilkan lima tugas harian (Home, Attendance, Scan, Team, Archive). Pelanggaran diurutkan paling atas pada daftar absensi; pekerja yang belum check-in memiliki aksi “Scan” satu ketukan; hasil scan menampilkan daftar periksa keyakinan per item dengan overlay kotak deteksi (Gbr. 1); status tidak pernah disampaikan hanya lewat warna. Tersedia tema terang dan gelap.



Gbr. 1. Hasil scan (kiri) dan tampilan absensi (kanan).

D. Penyimpanan dan pelaporan

Data karyawan dan scan disimpan sebagai JSON berseri pada penyimpanan kunci-nilai platform (seluler, desktop, dan web). Pembuat laporan menghasilkan PDF A4 bermerek berisi indikator ringkasan dan tabel per-scan untuk satu scan, satu pekerja, atau filter arsip saat ini.

V. IMPLEMENTASI DAN VERIFIKASI

Implementasi menargetkan Flutter 3.41 / Dart 3.11 dengan 32 berkas sumber ($\pm 4,9$ ribu baris) serta runner Android, iOS, web, Windows, dan macOS. Kode lolos `flutter analyze` tanpa error atau warning dengan aturan `very_good_analysis` (tersisa dua lint informasional). Rangkaian uji berisi tujuh tes: tiga uji unit (perhitungan kepatuhan dan pulang-pergi JSON; data awal deterministik; rentang keluaran detektor) dan empat uji widget menyeluruh (gerbang onboarding; menjalankan scan dan menyimpan pelanggaran; menambah karyawan; menyaring arsip).

Versi saat ini memakai detektor simulasi yang mengacak keyakinan item setelah jeda singkat. Detektor ini ada untuk

menjalankan seluruh alur tanpa kamera atau model terlatih; karena itu *makalah ini tidak mengklaim akurasi deteksi apa pun*. Prototipe terdahulu penulis (`ppe_apk_prototype`) telah menyertakan model YOLO; menghubungkannya di balik `PpeDetector` adalah langkah lanjutan terdekat.

VI. EVALUASI

[BELUM DIISI — isi minimum yang disarankan]

A. Benchmark detektor: dataset, pembagian data, hiperparameter, perangkat keras; presisi, recall, $mAP@0,5$ dan $mAP@0,5:0,95$ per kelas; latensi pada perangkat (ms/frame) dan memori pada ponsel yang disebutkan.

B. Studi kebergunaan: n partisipan; keberhasilan dan waktu tugas scan / menemukan pelanggaran / ekspor laporan; System Usability Scale [3]; protokol dan pernyataan etika.

C. Perbandingan alur kerja dengan daftar periksa kertas (waktu pencatatan).

VII. KETERBATASAN

Kepatuhan ditentukan per item dari satu frame; oklusi, pencahayaan, dan banyak pekerja dalam satu frame belum dimodelkan. Identitas dipilih dari daftar, bukan dikenali otomatis. Penyimpanan bersifat lokal pada perangkat. Pemantauan pekerja berbasis kamera menimbulkan isu privasi dan persetujuan; penerapan harus menetapkan kebijakan retensi dan persetujuan serta sebaiknya menyimpan skor, bukan gambar. **[BELUM DIISI: regulasi yang berlaku.]**

VIII. KESIMPULAN

Kami memaparkan GearGuard, arsitektur agnostik-detektor dan implementasi sumber terbuka untuk alur kepatuhan APD. Pekerjaan selanjutnya: detektor nyata pada perangkat di balik antarmuka yang sama, studi kebergunaan lapangan, dan pelaporan tingkat departemen.

DAFTAR PUSTAKA

1. J. Redmon, S. Divvala, R. Girshick, A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.
2. N. D. Nath, A. H. Behzadan, S. G. Paal, “Deep learning for site safety: Real-time detection of personal protective equipment,” *Automation in Construction*, vol. 112, 2020, Art. 103085.
3. J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, Taylor & Francis, 1996, pp. 189–194.

Verifikasi setiap referensi pada halaman penerbit sebelum pengiriman.